

Notes on rolling chunk windows

Matteo M. Costa

January 28, 2026

Abstract

This short summary describes the concept of rolling chunk windows for time series analysis. We define chunk windows, provide the mathematical formulation and explain the computation of aggregated statistics.

1 Introduction

A rolling chunk window is a type of temporal aggregation over fixed-length intervals of a time series. Unlike classical rolling windows, where the window moves one time step at a time, chunked windows divide the data into non-overlapping (or partially overlapping) blocks of length w and compute summary statistics for each block. This technique is widely used for:

- weekly or monthly reporting;
- aggregated metrics for time series analysis;
- reducing computational cost compared to fully overlapping rolling windows.

By summarizing data over fixed intervals, rolling chunk windows simplify analysis while preserving the essential temporal patterns.

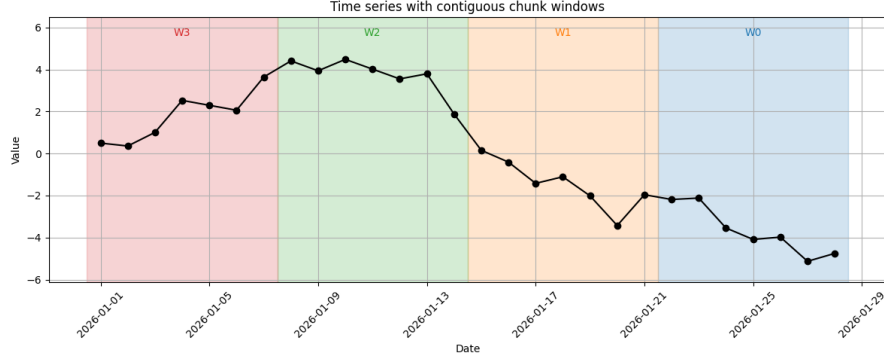


Figure 1: Time series of daily observations with rolling weekly chunk windows.

2 Mathematical formulation

Let $X = (X(t))_{t=1}^{T_{\max}}$ be a time series of observations indexed by a finite set of discrete time points. Let w be the fixed window length (number of days, weeks, or other time units). Define chunk windows starting from the maximum time $T_{\max}$ and moving backward as

$$W_i = \{X(t) \mid t \in \{T_{\max} - (i+1)w + 1, \dots, T_{\max} - iw\}\},$$

with $i = 0, 1, \dots, N$, where:

- $i = 0 \rightarrow$ the most recent window;
- $i = 1 \rightarrow$ previous window, and so on;
- each window contains exactly w observations (or fewer, depending on the available data).

In practice, a chunk window can be used to compute summary statistics over periods like weeks, months, or any other interval. For each chunk W_i , we can compute aggregated statistics:

$$\text{sum}_i = \sum_{t \in W_i} X(t), \quad \text{mean}_i = \frac{1}{|W_i|} \sum_{t \in W_i} X(t), \quad \text{max}_i = \max_{t \in W_i} X(t).$$

Other metrics can include variance, flags, or counts of conditions (e.g., $X(t) \geq 1$):

$$\text{count}_i = \sum_{t \in W_i} \mathbb{I}_{\{X(t) \geq 1\}},$$

where $\mathbb{I}_{\{\text{condition}\}}$ is an indicator function:

$$\mathbb{I}_{\{\text{condition}\}} = \begin{cases} 1 & \text{if condition is true} \\ 0 & \text{if condition is false} \end{cases}$$

2.1 General assumptions

When using rolling chunk windows for time series analysis, the following general assumptions are often made:

- fixed-length windows: each chunk window has a fixed length w , consisting of w consecutive observations. In cases where the time series does not contain enough data to complete the window, the window may be shorter than w . Such partial windows can be handled consistently depending on the analysis, for example by allowing partial windows, padding with missing values, or excluding them;
- blocks: the windows are typically non-overlapping, with each chunk starting immediately after the previous one ends. This means there is no overlap between consecutive windows;
- aggregated statistics: for each chunk window, aggregated statistics such as sum, mean, maximum, or count are calculated to summarize the data within that window;
- handling of missing data: missing data within a chunk is often handled by exclusion or imputation methods, depending on the specific analysis requirements.

These assumptions make the rolling chunk windows an efficient tool for periodic reporting and time series analysis.

2.2 Additional metrics

Other useful metric include:

- variance:

$$\text{var}_i = \frac{1}{|W_i|} \sum_{t \in W_i} (X(t) - \text{mean}_i)^2;$$

- median:

$$\text{med}_i = \text{median}\{X(t) \mid t \in W_i\};$$

- weighted mean:

$$\text{wmean}_i = \frac{\sum_{t \in W_i} w_i(t) X(t)}{\sum_{t \in W_i} w_i(t)},$$

with $w_i(t)$ is a weight associated with the observation $X(t)$ at time t within window W_i ;

- frequency of events (for binary flags):

$$\text{freq}_i = \frac{1}{|W_i|} \sum_{t \in W_i} \mathbb{I}_{\{X(t)=1\}};$$

- delta between consecutive windows:

$$\Delta \text{sum}_i = \text{sum}_i - \text{sum}_{i+1}.$$

3 A Python simulation

In this example, the dataset consists of 10 daily observations from January 1st to January 10th, 2026, with a weekly window of 7 days. As a result, the final window contains only 3 days, illustrating the case of a partial window, which can be handled according to the analysis needs (e.g., included as-is or excluded). The dataset includes several variables:

- *count_id_proc_corr*: integer, represents the count of successful procedures per day;
- *durata_tot*: total duration of procedures (in arbitrary time units) performed per day;
- *valore_tot*: total value (e.g., revenue, score, or metric) associated with procedures per day;
- *flag_count_gte_1*: binary flag indicating whether at least one procedure occurred on a given day (1 = yes, 0 = no).

Each row represents a single day and the dataset is sorted by the date column in ascending order:

```
1 # Import libraries
2 import pandas as pd
3
4 # Dataset
5 data = {
6     "date": pd.date_range(start = "2026-01-01", periods = 10, freq = "D"),
7     "count_id_proc_corr": [1, 0, 2, 1, 0, 3, 0, 1, 2, 0],
8     "durata_tot": [10, 5, 20, 15, 0, 25, 5, 10, 15, 0],
9     "valore_tot": [100, 50, 200, 150, 0, 250, 50, 100, 150, 0],
10    "flag_count_gte_1": [1, 0, 1, 1, 0, 1, 0, 1, 1, 0]
11 }
12
13 # Output
14 df = pd.DataFrame(data)
15 df
```

We implement a rolling window aggregation on the daily dataset to compute summarized statistics over fixed-length time windows. Specifically, we define a weekly window of 7 days (*window_days* = 7) and iterate backwards from the most recent date to the earliest date in the dataset. For each window, we select all observations within the date range and compute the following aggregated metrics:

- *count_id_proc_corr_sum / mean / max*: sum, average, and maximum of daily procedure counts;

- *durata_tot_sum / mean*: total and average duration of procedures per window;
- *valore_tot_sum / mean*: total and average value associated with procedures;
- *flag_count_gte_1_sum / mean*: total and average of the binary indicator that signals whether at least one procedure occurred.

Each window is stored as a row in a new dataframe *df_agg*, with a column *data_range* indicating the start and end dates of the window. This approach allows us to summarize temporal trends and behaviors over fixed periods:

```

1  # Rolling dictionary
2  rolling_map = {"weekly": 7}
3  window_days = rolling_map["weekly"]
4
5  # Sorting
6  df = df.sort_values("date")
7
8  # Max and min date
9  max_date = df["date"].max()
10 min_date = df["date"].min()
11
12 agg_list = []
13
14 # Rolling loop
15 while max_date >= min_date:
16     start_date = max_date - pd.Timedelta(days = window_days - 1)
17     start_date = max(start_date, min_date)
18
19     mask = (df["date"] >= start_date) & (df["date"] <= max_date)
20     df_window = df.loc[mask]
21
22     if not df_window.empty:
23         agg_dict = {
24             "data_range": f"{start_date.date()}-{max_date.date()}",
25             "count_id_proc_corr_sum": df_window["count_id_proc_corr"].sum(),
26             "count_id_proc_corr_mean": df_window["count_id_proc_corr"].mean(),
27             "count_id_proc_corr_max": df_window["count_id_proc_corr"].max(),
28             "durata_tot_sum": df_window["durata_tot"].sum(),
29             "durata_tot_mean": df_window["durata_tot"].mean(),
30             "valore_tot_sum": df_window["valore_tot"].sum(),
31             "valore_tot_mean": df_window["valore_tot"].mean(),
32             "flag_count_gte_1_sum": df_window["flag_count_gte_1"].sum(),
33             "flag_count_gte_1_mean": df_window["flag_count_gte_1"].mean()
34         }
35         agg_list.append(agg_dict)
36
37     max_date = start_date - pd.Timedelta(days = 1)
38
39 # Output
40 df_agg = pd.DataFrame(agg_list)
41 df_agg

```

Clearly, this code example can be generalized into a function with useful parameters. Another interesting improvement could be a dynamic implementation of the rolling dictionary: in this way, it is possible to handle different cases.

Conclusion

Rolling chunk windows provide an effective method for aggregating and analyzing time series data over fixed intervals. By assuming fixed-length, non-overlapping windows, we simplify the process of summarizing and interpreting temporal trends while maintaining computational efficiency. These assumptions are commonly used in practice for time series reporting, trend analysis, and smoothing techniques.